23 最小生成樹

Part VI Graph Algorithms

本章定位

最小生成樹以割上的安全邊逐步降低連接成本；本章從交換證明推導Kruskal與Prim，再用同一五點圖追蹤分量、key及parent，建立可獨立檢查的最優與唯一性證書。

前置知識

連通無向圖、cut 與權重。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用 cut 性質推導 MST 安全性。
- 比較 Kruskal 與 Prim 的資料結構與成本。
- 辨識負權、非連通與唯一性條件。
- 23.1 Growing a minimum spanning tree
- 23.2 The algorithms of Kruskal and Prim

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

23.1 Growing a minimum spanning tree

目標是樹的總權，並非每一條路的長度

給有限無向圖G=(V,E)，n=|V|、m=|E|；每條邊有可比較的有限實數權w(e)。先假設n≥1且G連通。Spanning tree（生成樹）T使用全部頂點、連通且無cycle，恰有n−1條邊；MST使$w(T) = \sum_{e \in T} w(e)$最小。有限個候選樹保證最小值存在，權可以為負、為零或重複。

負權不會破壞MST，但不能刪去「無cycle」條件：三角形三邊皆−1，樹成本−2；若容許一般連通子圖，取三邊成本−3。MST也不是最短路徑樹：在ab:4、ac:5、bc:2中，MST={ab,bc}成本6，a到c沿樹長6；最短路徑卻是ac長5。

n=1時回傳空邊集合、成本0；n=0約定空forest。不連通圖沒有生成樹，minimum spanning forest（MSF）則在每個原連通分量取一棵MST。自環不屬於任何樹；平行邊保留各自edge ID，不能僅以端點辨識刪除對象。

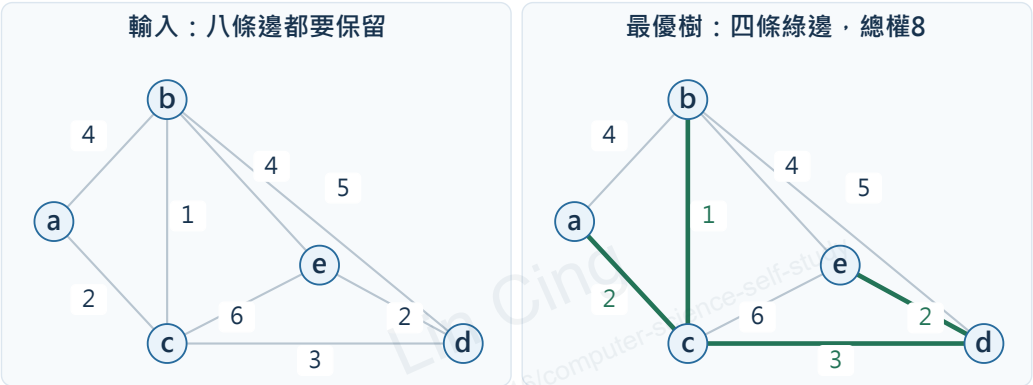
同一五點圖與兩種不同成長順序

本章固定邊表如下，所有後續狀態皆使用同一輸入。Kruskal同權依edge ID先後處理。

ID	0	1	2	3	4	5	6	7
邊	ab	ac	bc	bd	cd	ce	de	be
權	4	2	1	5	3	6	2	4

Kruskal按權選bc、ac、de、cd；Prim從a選ac、cb、cd、de。兩者最後都是{bc,ac,de,cd}，成本1+2+2+3=8，但中途的forest與grown set不同。圖中的灰邊也屬於輸入，不能只看最後綠樹就猜出選取過程。

同一五點圖與兩種不同成長順序 （續）



bc:1 + ac:2 + de:2 + cd:3 = 8

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Safe 的意思是仍能延伸成某棵最優樹

維護不變量：目前A⊆E包含在至少一棵MST內。e∉A稱為safe for A，若A∪{e}也包含在某棵MST內。這是最優性的延伸條件，比「加入不成cycle」強；後者只保證還是一片forest。

Cut（割）(S,V−S)要求∅≠S≠V；邊的兩端分居兩側才叫crossing。割respect A表示沒有A中的邊跨割。Light edge是該割跨邊中的最小權者，可以並列。

通用算法由A=∅開始，反覆找safe edge加入，直到|A|=n−1。每次加一條且不變量保證A無cycle；n點n−1邊的forest必連通，所以回傳MST。問題在於如何有效地證明下一條邊safe，不能把「每次選便宜邊」當作證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

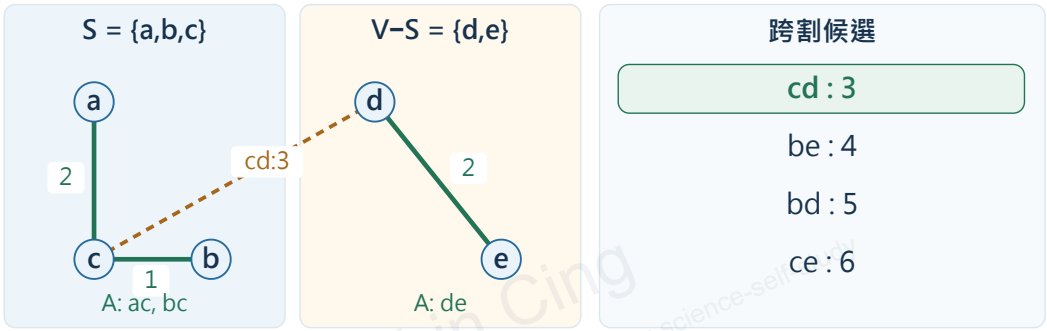
安全邊定理：交換時為何不會丟掉已選的邊

設A可延伸成MST，割(S,V−S)respect A，e是此割的一條light edge。取含A的MST T。若e∈T便已成立；否則T+e恰形成一個cycle。e跨割，cycle要回到起側必另經跨邊f。因割respect A，f∉A。

刪f後cycle被打斷且仍連通，所以T′=T−{f}+{e}仍是生成樹。由w(e)≤w(f)，w(T′)≤w(T)；T已最優，故T′也是MST，且A∪{e}⊆T′。這同時交代結構合法、權不增加、A不受損三件事。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

安全邊定理：交換時為何不會丟掉已選的邊 （續）



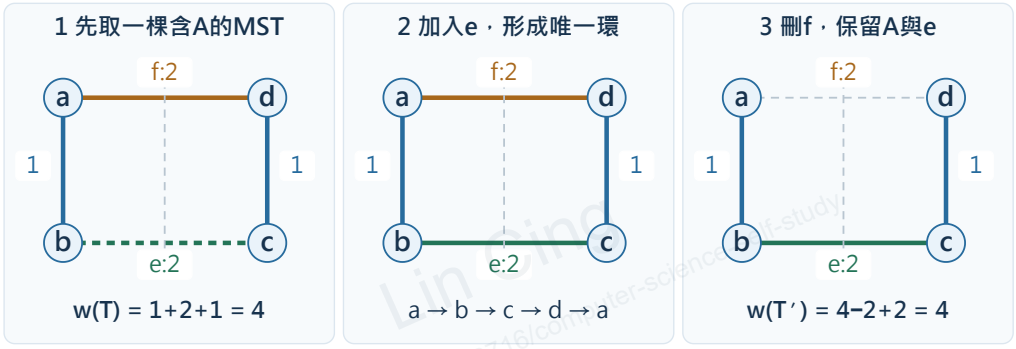
A內三條邊都沒有跨割；cd是最輕跨邊。

respect A：保留已選邊，再加入安全邊 cd

此例S={a,b,c}，A={ac,bc,de}；cd:3、be:4、bd:5、ce:6跨割，故cd safe。de位於另一側內部，並沒有跨割。

交換的三個時刻：一條邊進來，另一條出去

四點圖：ab:1、cd:1、ad:2、bc:2、ac:4、bd:5。A={ab,cd}，割為{a,b}｜{c,d}。



藍邊 A={ab,cd} 始終保留；f不在A中。綠虛線是待加入的e。

同權交換：最優值不變，但找到一棵包含e的MST

T用三條邊達到權重下界1+1+2=4。加入e=bc後，沿a-b-c-d-a的環刪去f=ad，成本4不變且保留A。**只保證存在含A與e的MST；原T就沒有e。**

並列、必選與cycle性質的量詞

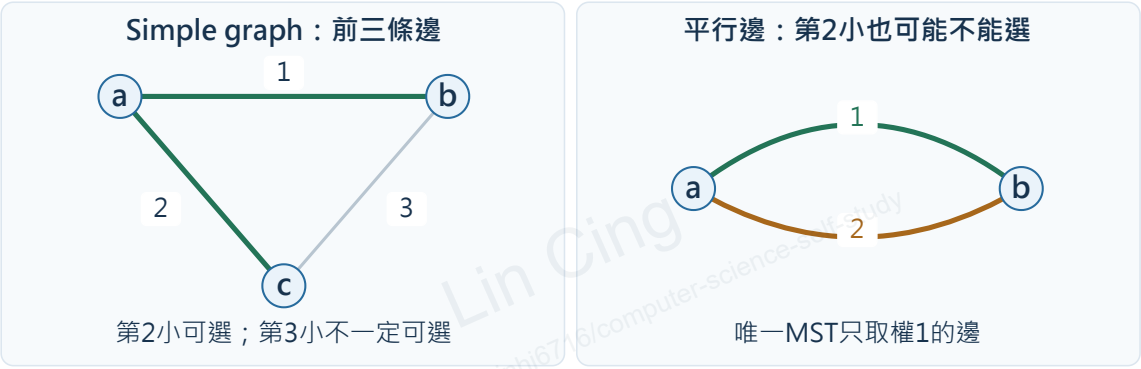
若e是某割的**唯一**light edge，任何不含e的MST都可用上述交換嚴格降低成本，矛盾；所以每棵MST都含e。若只是並列light，只能保證至少有一棵MST含e。三角形各邊權1時有三棵MST，每條邊都是某割的light，卻不能把三條一併選進樹。

Cycle性質：若e是某cycle的唯一最重邊，則e不屬於任何MST。假若MST T含e，刪e分成割；原cycle去e的路必有另一跨邊f，且w(f)<w(e)，交換便嚴格改善。若e只是並列最重，仍可證「存在一棵MST不含e」：從含e的MST做不增成本交換；不能說e一律不可選。

Forest A中任一連通分量C都給出respect A的割(C,V−C)。因此C對外最輕邊safe；這是Kruskal與Prim共同的依據，不是要求每輪全圖最小邊。

全圖第2小邊：結論取決於圖的模型

連通simple graph至少有兩條邊，按非下降權重排序，同權任意排。第2條e必屬某棵MST。



第2小邊的結論要求連通、無自環且無平行邊

若MST T未含e，T+e的環長至少3。除e外至少兩條邊，而全圖嚴格輕於e的邊至多一條，故環上有f不比e輕。刪f後仍最優，得到含e的MST。**平行邊使「環長至少3」失效；第3小邊也沒有相同保證。**

23.2 The algorithms of Kruskal and Prim

Kruskal：讓分量依全域權序合併

```
KRUSKAL(V, E):
    A = 空集合
    對每個 v∈V: MAKE-SET(v)
    將 E 按 (weight, edge-ID) 非下降排序
    對排序後每條 e=(u,v):
        ru = FIND(u); rv = FIND(v)
        若 ru != rv:
            A.add(e); UNION(ru,rv)
    回傳 A
```

DSU代表目前(V,A)的分量，不是原圖的最終連通分量。相同root表示已有一條A路徑，加入e會形成cycle（自環亦在此被拒）。不同root時兩棵樹用一邊連起，仍無cycle。連通輸入接受n−1邊即可提前返回；不必再掃剩餘cycle邊。若要支援MSF，亦可掃到輸入結束。

Kruskal 的一般正確性：被略過的便宜邊去了哪裡

假設接受前A可延伸成MST。對下一條接受的e=(u,v)，令S為u所在的當前DSU分量。S割respect A；若有比e更早的跨邊f，它早已處理過。當時若接受，兩端會合併且此後只合不分，不可能目前仍跨S；當時若拒絕，兩端當時已同分量，現在也仍同分量，也不可能跨S。因此沒有較早跨邊。

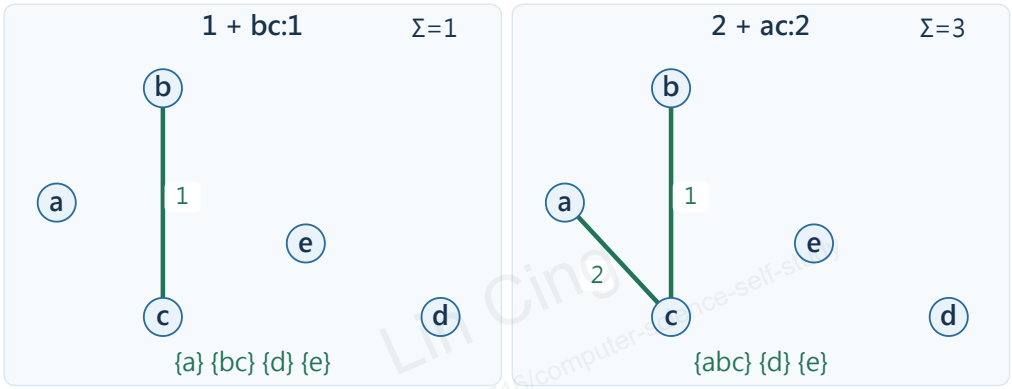
所有尚未掃的邊權不小於e，所以e是這一割的light edge。安全邊定理維持最優延伸不變量。結束時若原圖某分量仍被分成至少兩個A分量，原圖連通路上必有跨A分量邊；掃它時不會被拒，矛盾。故每個原分量都得到生成樹且最優，即MSF。

這個證明不假設權重皆不同，也不假設DSU root剛好是最小頂點；root是實作細節。

Kruskal 真正的每一步

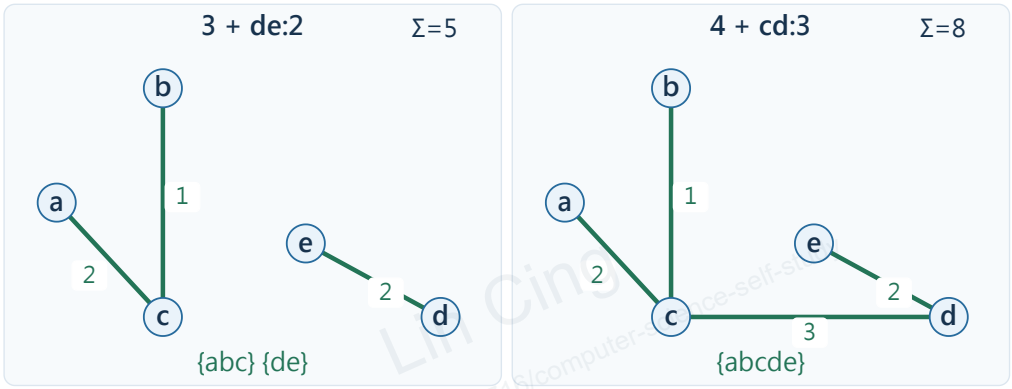
掃描	兩端原分量	決定	處理後分量	累計成本
bc:1	{b},{c}	接受	{bc},{a},{d},{e}	1
ac:2	{a},{bc}	接受	{abc},{d},{e}	3
de:2	{d},{e}	接受	{abc},{de}	5
cd:3	{abc},{de}	接受	{abcde}	8
ab:4	同一分量	拒絕	不變	8
be:4	同一分量	拒絕	不變	8
bd:5	同一分量	拒絕	不變	8
ce:6	同一分量	拒絕	不變	8

Kruskal 真正的每一步 (續)



僅畫已選邊；框下的集合列出所有分量

先長出兩棵樹，再用一條邊合併



第3輪de另成一棵樹；Kruskal先長出forest

第4步已取n−1=4邊，可直接停止；後四列只用來核對完整掃描版本。第3步的de雖不接到{abc}，仍是合法的forest成長，這與Prim不同。

Kruskal 的成本與圖表示前提

比較排序O(m log(m+1))；DSU以union by rank及path compression處理n次MAKE-SET、2m次FIND、至多n−1次UNION，總O((n+m)(1+α(n)))。依第21章固定的版本，α(2)=0；只有n≥3時才能將1+α(n)簡寫成O(α(n))。加上讀入，完整成本為O(m log(m+1)+(n+m)(1+α(n)))；空圖直接返回空森林。排序邊表O(m)空間，DSU O(n)，輸出O(n)。

對連通**simple graph**，n−1≤m<n^2，log m=O(log n)，因此常寫O(m log n)。若允許任意多平行邊，不能再用m<n^2，必保留log m或先用額外演算法去除冗餘邊。負權與同權不影響比較排序。

整數權落在0..W時可用counting sort取代比較排序，成本O(m+W)，但DSU成本仍要加上；W很大時不會更好。這是改變鍵表示的假設，並非推翻比較排序下界。

Prim：key 是跨割的一條邊，不是路徑距離

設已取頂點S。每個v∉S維護$key[v] = \min_{u \in S, (u,v) \in E} w(u,v)$，沒有此邊則∞；parent[v]記實現該值的S端點及edge ID。所有未取頂點各有一個priority-queue項目。第一根r的key特設0，只為先取出，不能把0當輸出樹邊權。

每輪extract最小key。除第一根外，最小項u的parent邊恰是割(S,V−S)的light edge，所以可加入。取u後只需把新出現的候選w(u,v)與key[v]比較，不是key[u]+w(u,v)。

在ab:4、ac:5、bc:2例，取a後key[b]=4、key[c]=5；取b後key[c]=2，但a到c沿樹長4+2=6。Dijkstra則比較d[b]+2=6，保留d[c]=5。Prim允許負權；Dijkstra的正確性另需非負邊。

Prim 的完整 queue 契約

```
PRIM-CONNECTED(V, Adj, r):
    對每個 v: key[v]=∞; parent[v]=NIL; inQ[v]=true
    key[r]=0; Q=BUILD-INDEXED-MIN-HEAP(V,key)
    A=空集合
    當 Q 非空:
        u=EXTRACT-MIN(Q); inQ[u]=false
        若 key[u]==∞: 回報輸入不連通
        若 parent[u]!=NIL: A.add(parent[u]所記的邊)
        對 Adj[u]每條 e=(u,v):
            若 inQ[v] 且 w(e)<key[v]:
                parent[v]=(u,e.ID)
                DECREASE-KEY(Q,v,w(e)); key[v]=w(e)
    回傳 A
```

DECREASE-KEY視為同步更新heap與key儲存。inQ用陣列O(1)查，不可每次掃heap找會員。要輸出MSF時，extract∞代表新原分量：將該u設新根、key[u]=0、不加parent邊，再正常掃鄰居；不能把∞當樹成本。

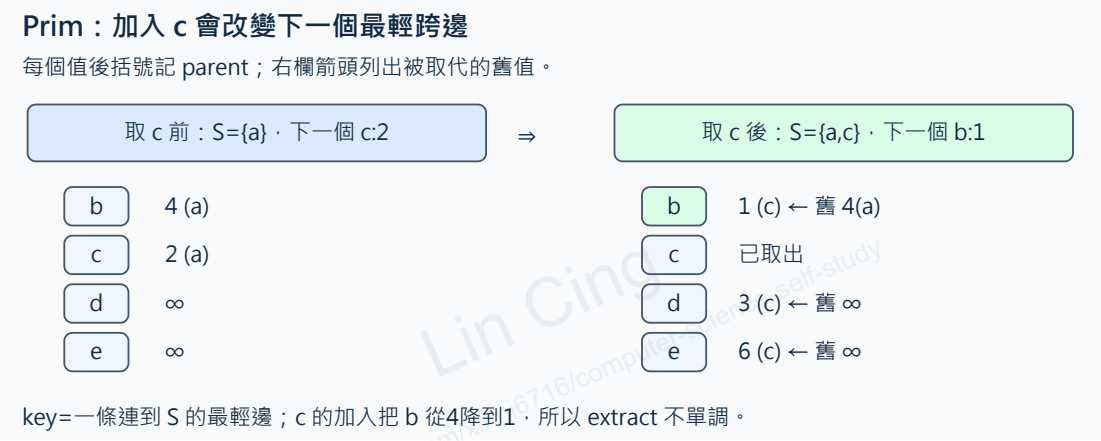
Prim 的不變量、終止與完整數值

每輪開始，S=V−Q，A在S上為一棵樹（第一輪前為空），且包含在某MST中；對Q中各點，key與parent恰表示至S最輕單邊。初始化以根的特例成立。extract後用割定理加入安全邊；掃u的鄰接串列恰補足所有由新點產生的跨邊候選，故不變量恢復。每次Q少一點，n輪終止，輸出n−1邊MST。

本輪extract	掃完後仍在Q的 key(parent)	已接受邊	成本
a:0	b4(a), c2(a), d∞, e∞	無	0
c:2	b1(c), d3(c), e6(c)	ac	2
b:1	d3(c), e4(b)	ac,cb	3
d:3	e2(d)	ac,cb,cd	6
e:2	空	ac,cb,cd,de	8

key extract序列0,2,1,3,2不是遞增。新點能提供更便宜跨邊；最優性靠cut而非key單調。

從新候選到下一條安全邊



圖左S={a}時ac:2最輕；取c後，b的候選由ab:4降成cb:1，d與e才第一次有有限key。下一步必取b:1，不能因「剛取過2」而排除權1。b被取後be:4取代ce:6；再取d才由de:2取代be:4。

每次parent更新要和同輪key一起保存；最後parent不是「第一次碰到的鄰居」。只把頂點順序列出，無法重建這些競爭候選。

Prim 的實作成本不能混用

queue／圖表示	工作拆解	總成本
indexed binary heap＋adjacency list	n次extract、至多2m次掃邊與m次decrease	$O((n+m)\log n)$
Fibonacci heap＋adjacency list	extract共$O(n \log n)$攤銷、decrease共$O(m)$攤銷	$O(m+n \log n)$
線性找最小key＋adjacency list	n輪各掃最多n個key，掃邊共2m	$O(n^2+m)$
adjacency matrix＋線性key	n輪各掃n個key與n個鄰位	$\Theta(n^2)$

對simple graph，$m=O(n^2)$，第三列亦$O(n^2)$。n=1的heap邊界直接返回空樹。heap需indexed handle讓decrease定位O(1)；改成每次push新項、取出時丟舊項的lazy版本，項目數可到O(m)，不可直接套只有n個項目的證明。

$m=\Theta(n)$時binary與Fibonacci均為$O(n \log n)$；$m=\Theta(n^2)$時兩者分別$O(n^2 \log n)$、$O(n^2)$，線性掃key也$O(n^2)$。實際例程用dense Prim驗算選邊，不假裝測量過Fibonacci速度。

不重跑算法也能檢核一棵候選樹

設T是生成樹。以下證書只量化**非自環的非樹邊**e=(u,v)、u≠v，令M_T(u,v)為T中唯一u-v路的最大邊權。自環永不能入樹，直接忽略；不對空的u-u路定最大邊。則

$$T \text{ 是MST} \iff \forall e=(u,v)\in E\setminus T,\ u\neq v:\ w(e)\geq M_T(u,v).$$

必要性：若某w(e)<M，加入e並刪路上最大邊f，仍是樹且嚴格便宜，矛盾。充分性：對任一f∈T，刪f形成割；其他跨邊e都不在T，且T上的e端點路經f，所以w(e)≥M_T≥w(f)。故f為此割light edge。依任意順序加入T的邊時，該割沒有其他T邊跨越，必respect已選prefix；反覆用安全邊定理，整棵T最優。

同一圖非樹邊ab、bd、ce、be的(w,M)依序為(4,2)、(5,3)、(6,3)、(4,3)，全部嚴格大於。因此此例MST唯一。

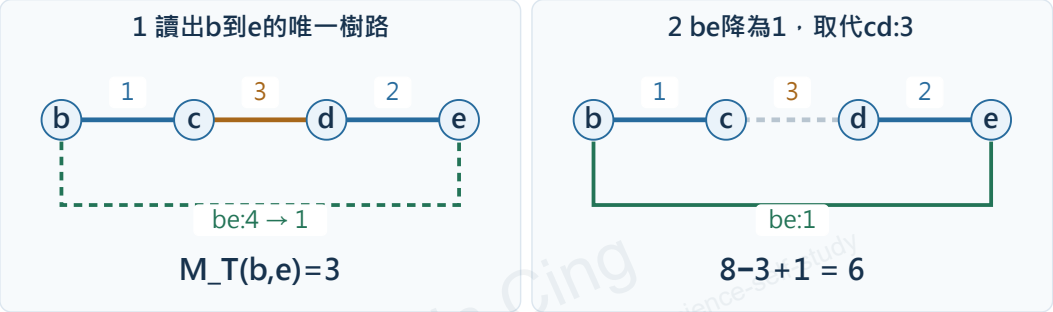
唯一性與邊權變動：證書的兩個用途

T唯一當且僅當每條非自環的非樹邊都有w(e)>M_T。若等號成立，交換一條最大路徑邊得到另一棵同成本樹。反之若全嚴格，前述每個T−f割上，f是唯一light edge，故每棵MST都含全部T邊。這不要求**所有輸入邊權互異**；本章兩條權2仍得到唯一MST。

若只將一條非自環的非樹邊e的權降成w′，其他不變，令M為原T路徑最大值。w′≥M時原T仍通過全部證書；w′<M時刪一條最大邊f並加e。其最優值為w(T)+min(0,w′−M)：不含e的樹權重未變，至少w(T)；任一含e的樹R，刪e分割後，T中u-v路有跨邊g；R−e+g是原權的樹，故w(T)≤w(R)−w′+w(g)≤w(R)−w′+M。這給與新構造相同的下界。

本圖把be由4降到1：T路b-c-d-e權1,3,2，刪cd:3，成本8−3+1=6。若只降到3，舊樹仍最優且出現並列。逐條DFS找路的證書成本O(nm)；這是單邊權變動的更新保證。自環無論多負都不能入樹，改其權不影響MST，亦不執行刪除樹路最大邊。

唯一性與邊權變動：證書的兩個用途 (續)



ac:2保持不變；圖只放大發生交換的b-c-d-e路徑。

新邊比路上最大權更小，才需要替換：$w' < M_T$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 23。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study